Astronomy & Astrophysics manuscript no. paper
October 27, 2026
©ESO 2026

Synthesising full-sky maps from scattering covariances on distributed-memory machines

A pyramidal, block-wise explicit gradient in Rust, MPI and CUDA, applied to weak-lensing convergence

J.-M. Delouis[1]⋆ and [TODO: co-authors]

Laboratoire d'Océanographie Physique et Spatiale (LOPS), Univ. Brest, CNRS, Ifremer, IRD, Brest, France

Received ...; accepted ...

ABSTRACT

Context. Scattering-covariance statistics generate new realisations of a non-Gaussian field from one or a few examples with a small, interpretable set of constraints. On the sphere they are limited by the memory of automatic differentiation: a full-sky gradient at $N_{\rm side} = 2048$ (5×10^7 pixels) fills a large GPU.
Aims. We make the synthesis scale with the nodes of a distributed-memory machine, each process holding a memory bounded by its share of the sky, and we test syntheses of a strongly non-Gaussian field against independent realisations.
Methods. The statistics and their exact gradient are computed without a computational graph, block by block, on a pyramid of anti-aliased HEALPix decimations; the cross-scale terms are carried by decimated moduli stored once per pair of levels, and blocks interact only through fixed halos. The Rust implementation uses MPI (contiguous NESTED ranges, halo exchanges, additive ghost cells, a distributed L-BFGS) and one CUDA device per rank, and matches the reference Python code to 10^{-15}. For weak-lensing convergence the loss adds, in a gaussianised space, the first-order statistics of κ, the Wasserstein distance of the histogram, the power and the kurtosis of every pyramid level, and the scatter of an ensemble.
Results. The evaluation time drops as the inverse of the number of ranks with 75–85% efficiency up to 512 CPU cores at $N_{\rm side} = 1024$, with the same losses to 16 digits for any rank count. Ensembles of ten syntheses of the Takahashi et al. (2017) convergence map at $N_{\rm side} = 1024$ lie within the scatter of ten independent realisations for the angular power spectrum, the per-level variance of κ and the peak counts, but not for the Minkowski functionals of the finer levels. [TODO: 2048 results.]
Conclusions. Scattering-covariance synthesis is no longer bound to the memory of one device, and the same machinery applies to high-resolution, multi-channel and time-dependent geophysical fields such as climate and ocean model outputs.

Key words. Methods: numerical – Methods: statistical – gravitational lensing: weak – large-scale structure of Universe – Techniques: image processing

1. Introduction

Many of the largest numerical data sets of the physical sciences are fields on the sphere. Kilometre-scale global climate and ocean models produce outputs of 10^7 to 10^9 cells per level and per variable, for many variables, every few hours over decades (Stevens et al. 2019; Eyring et al. 2016; Hewitt et al. 2020); reanalyses such as ERA5 hold several decades of hourly global fields (Hersbach et al. 2020); and the weather forecasts produced by machine-learning models are now judged as probabilistic ensembles of global fields (Lam et al. 2023; Bi et al. 2023; Price et al. 2025; Kochkov et al. 2024), some of them on HEALPix grids (Karlbauer et al. 2024). In cosmology, wide surveys measure the weak-lensing convergence and the CMB on the full sky at arcminute resolution, and their non-Gaussian statistics are the target of a growing body of work (Kratochvil et al. 2010, 2012; Petri et al. 2013; Euclid Collaboration: Ajani et al. 2023; Gatti et al. 2024). All these applications need new realisations of the fields: to estimate covariances, to propagate uncertainties, to test pipelines, or to augment a small set of expensive simulations.

Generative models based on scattering transforms (Mallat 2012; Bruna & Mallat 2013) and their covariances (Morel et al. 2025; Cheng et al. 2024) address this need with a different trade-off from deep generative networks. A scattering-covariance model describes a field by a few thousand averaged, interpretable coefficients that capture the coupling between scales and orientations; new realisations are drawn by gradient descent from noise until their coefficients match those of the data, in the spirit of microcanonical models (Bruna & Mallat 2018) and of texture synthesis (Portilla & Simoncelli 2000; Gatys et al. 2015). Such models need no training set: one realisation is enough (Allys et al. 2019, 2020; Cheng et al. 2020; Cheng & Ménard 2021; Jeffrey et al. 2022; Regaldo-Saint Blancard et al. 2021). They have been brought to the sphere for Planck dust polarisation (Delouis et al. 2022), for astrophysical fields at large (McEwen et al. 2022; Mousset et al. 2024), and to emulate CMB systematics from a handful of simulations (Campeti et al. 2025).

The obstacle to larger maps is memory. The synthesis needs the gradient of the statistics with respect to every pixel. With automatic differentiation the computational graph keeps every intermediate field: on the sphere, the wavelet transforms of every scale and orientation, their moduli, and the second wavelet trans-

⋆ Corresponding author: jean.marc.delouis@ifremer.fr

forms of the moduli. The Python engine of STL-Dev[1] (STL-Proj 2025) replaced the graph by an explicit, pyramidal and block-wise gradient, and synthesises an $N_{\rm side} = 2048$ map on one 24 GB GPU. Beyond that size, and for several channels or for time sequences, one device is not enough, whatever the implementation.

This paper presents STL-Dev-MPI, a port of that engine to distributed-memory machines: Rust for the core, MPI for the decomposition of the sky between processes, and CUDA for one device per process. The memory of each process is bounded by its share of the sky; the wall-clock time per evaluation drops with the number of processes; and every rank count gives the same result to the last digits. We describe the method (Sect. 2), its parallel implementation on CPUs (Sect. 3) and GPUs (Sect. 4), and the terms we had to add to the loss to synthesise the weak-lensing convergence, a field whose information sits in rare, extended peaks (Sect. 5). We then test ensembles of ten syntheses of the Takahashi et al. (2017) convergence map against the scatter of ten independent realisations of the same simulation suite, with statistics that are not in the loss: the angular power spectrum, the per-level variance and kurtosis, the peak counts, the one-point PDF and the Minkowski functionals (Sect. 6). Section 7 discusses what the distributed implementation opens, in particular for climate and ocean data.

2. Method: a pyramidal, block-wise explicit gradient

2.1. Scattering covariance on a HEALPix pyramid

We use the scattering-covariance statistics of STL-Dev, which follow Morel et al. (2025) and Cheng et al. (2024) and were adapted to HEALPix (Górski et al. 2005) by Delouis et al. (2022) and Mousset et al. (2024); we only recall the notation. Let x be a map at resolution $N_{\rm side}$ (level 0) with C channels, and $x_d = D^d x$ its anti-aliased decimations, level d at $N_{\rm side}/2^d$. One decimation step D gives each coarse pixel a Gaussian-weighted, ℓ^1-normalised mean of the fine pixels whose centres lie within $r = 3\,\theta_{\rm fine}$ of its centre ($\theta_{\rm fine}$ the fine pixel size, Gaussian width $r/2$). The wavelet ψ is a complex directional filter with L orientations, applied at every level with the same $K \times K$ stencil in the tangent plane (taps interpolated bilinearly from the four nearest pixels): $z_d = \psi \star x_d$ is a complex field of shape $[C, L, N_d]$, and scale j is computed at level $d(j) = j$. Writing $\langle\cdot\rangle_n$ for a mean over the pixels of level n, the coefficients are

$$S_1(j) = \langle |z_n| \rangle_n, \qquad S_2(j) = \langle |z_n|^2 \rangle_n, \tag{1}$$

$$S_3(j_1, j_3) = \langle [\psi \star m_{d_1,n}]_{\ell_1 \ell_3}\, z^*_{n,\ell_3} \rangle_n, \tag{2}$$

$$S_4(j_1, j_2, j_3) = \langle [\psi \star m_{d_1,n}]_{\ell_1 \ell_3}\, [\psi \star m_{d_2,n}]^*_{\ell_2 \ell_3} \rangle_n, \tag{3}$$

with $n = d(j_3)$ ($n = d(j)$ for S_1, S_2), $d_i = d(j_i)$, cross-channel versions of each, the mean and variance of the map, and the *stored moduli*

$$m_{d,n} = D^{n-d}\,|z_d| \qquad (d < n), \tag{4}$$

the modulus of the wavelet field of level d decimated to level n. Every coefficient is a pixel mean at its *native level* n. With $L = 4$, $K = 5$ and $J = \log_2 N_{\rm side} - 1$ scales, a single-channel map has 11 354 coefficients at $N_{\rm side} = 1024$ and 15 042 at 2048.

The synthesis minimises over the map x

$$\mathcal{L}_S(x) = \frac{1}{N_c} \sum_i \frac{|S_i(x) - S_i(t)|^2}{\sigma_i^2}, \qquad \sigma_i^2 = \frac{\langle |f_i|^2 \rangle - |S_i(t)|^2}{N_{n(i)}}, \tag{5}$$

with t the target, f_i the field whose mean is S_i, and N_n the number of pixels of level n: σ_i^2 is the variance of a mean of N_n independent pixels, estimated on the target (ergodic hypothesis), and $\mathcal{L}_S$ is a reduced χ^2. Without these weights the loss is dominated by the low-order coefficients and S_3, S_4 stay at 1–3σ after 400 iterations; with them every family converges below 1σ.

2.2. Block decomposition of the cross-scale terms

The memory of a gradient computed by automatic differentiation is set by the cross-scale terms: S_3 and S_4 at level n read the moduli of every finer level $d < n$, so the graph keeps $|z_d|$ for every d and every orientation until the coarsest level is done. The pyramid makes the cost of the stored moduli themselves small: summed over the pairs $d < n$, Eq. (4) holds about $\frac{4}{9}N$ values per channel and orientation for a map of N pixels. The point is to compute them without ever holding the full-resolution fields. Since D is linear,

$$m_{d,n} = \sum_k D^{n-d}\left(|z_d|\,\mathbf{1}_{F_k}\right), \tag{6}$$

where $\{F_k\}$ is a partition of level d into blocks of b pixels. Each *piece* needs z_d on F_k only, which needs x_d on F_k plus a convolution halo, and its decimation chain is supported on a cone that grows by a decimation halo per level. The gradient goes back through the transposed pieces, the derivative of the modulus and the adjoint of the wavelet convolution.

Halos are counted in pixels of their level: $(K-1)/2 + 2$ for a convolution, $\lceil r/\theta \rceil + 1$ for a decimation, doubled for the pieces and their transposes so that every coarse row touched by a block sees its whole cone. Fine-level dilations are computed at the parent level (k coarse rings cover at least $2k$ fine pixels). The blocks are NESTED ranges, compact on the sphere, so a halo is a thin ring around a patch.

2.3. Sweeps

One evaluation of the loss and of its gradient is two sweeps over the levels.

Upward sweep, $n = 0, \ldots, J-1$: for every block F of level n, (i) the wavelet field z_n on F plus a halo, from x_n on F plus two halos; (ii) the second wavelets of $|z_n|$ (same level) and of $m_{d,n}$ ($d < n$, read from the stored maps) on F; (iii) the partial sums of every coefficient of native level n; (iv) the piece $|z_n|\mathbf{1}_F$ pushed down to every coarser level $n' > n$ and added to $m_{n,n'}$.

Downward sweep, $n = J-1, \ldots, 0$: for every block of level n, the fields of the upward sweep are recomputed; the coefficient cotangents $2(S_i - S_i(t))/(N_c\sigma_i^2)$ (real and imaginary parts for complex coefficients) are applied; the cotangents $g_{d,n}$ of the stored moduli of the finer levels are accumulated (they are complete once the coarser levels are done); the block's own chain is reversed from $g_{n,n'}$, $n' > n$, through the transposed pieces; then the modulus derivative and the wavelet adjoint give the cotangent of x_n on the block.

Finally the map cotangents of all levels are brought back to level 0 by Horner's scheme through the transposed decimations, $\nabla_x \mathcal{L} = \sum_d (D^\top)^d\, g_{x_d}$. Everything is in double precision and every summation order is fixed.

2.4. Memory

Per block the engine holds fields of size $|F + \text{halo}|$ times CL, and the convolution geometry of the block's rows: the rotated

[1] https://github.com/STL-Proj/STL-Dev

Table 1. Agreement of STL-Dev-MPI with the STL-Dev reference.

Quantity	Max. difference
Pixel indices, neighbours, decimation sparsity	exact
Pixel centres, stencil rotations	10^{-14}
Ring interpolation, decimation weights	10^{-12}
Block wavelet, adjoint, pieces	8×10^{-14}
Statistics (values $O(1)$)	7×10^{-16}
Map and band gradients	9×10^{-16}
Gradient vs finite differences (relative)	2.5×10^{-9}
2, 4 ranks vs 1 (threads, `mpirun`)	7×10^{-16}
GPU vs CPU statistics	6×10^{-16}
L-BFGS losses vs STL-Dev[a]	2.3×10^{-10}

Notes. [a] $N_{\rm side} = 64$, 25 evaluations.

stencil of each output row, interpolated on the four nearest pixels, $4GK^2$ entries per row for G gauges, computed on the fly and never stored globally (a stored geometry costs kilobytes per pixel). Globally it holds the maps of all levels and their cotangents ($2 \times \frac{4}{3}NC$ values), the stored moduli and their cotangents ($2 \times \frac{4}{9}NCL$, real), and the history of L-BFGS ($2(h+2)N$ values for a history h, work vectors included). For B maps synthesised together, every term is multiplied by B. At $L = 4$ and $C = 1$ the maps, the stored moduli and their cotangents take $8(8/3 + 8L/9) \simeq 50$ bytes per pixel and map before the halos (0.6 GB at $N_{\rm side} = 1024$, 2.5 GB at 2048), and L-BFGS with $h = 10$ adds 192 bytes per pixel and map. The GPU engine (Sect. 4) also keeps the level fields z_n and $|z_n|$ whole, about 220 bytes per pixel and map in all (2.8 GB at $N_{\rm side} = 1024$, 11.1 GB at 2048). The point of the decomposition (Sect. 3) is that all these terms are divided by the number of processes.

2.5. Validation against the reference

STL-Dev-MPI was written against fixtures dumped from STL-Dev; every test compares the two (Table 1). The geometry agrees to 10^{-12} or to the rounding of the reference itself (its `acos` differs from the C library's by one ulp on about 10% of the inputs, amplified by $1/\sin\theta$ near the poles). The statistics agree to 7×10^{-16} and the gradients to 9×10^{-16}, over 8 cases with $N_{\rm side} = 16$ and 32, $J = 3$ and 4, $L = 2$ and 4, kernels 3 and 5, 1 and 2 channels, both definitions of S_3/S_4, and blocks from 64 pixels to whole levels. The L-BFGS optimiser is a line-by-line port of `torch.optim.LBFGS` (Liu & Nocedal 1989; Paszke et al. 2019) with its strong Wolfe line search (Wolfe 1969; Nocedal & Wright 2006): a 20-iteration synthesis of the WebSky convergence map at $N_{\rm side} = 64$ reproduces the 25 loss values of STL-Dev at the printed precision, with any number of ranks.

3. Distributed memory

3.1. Decomposition and halos

The sky at level 0 is split into contiguous NESTED ranges, one per rank, aligned on 4^k pixels with k as large as the rank count allows. A rank owns the cell c of level d when its first descendant at level 0, $c\,4^d$, is in its range. When the ranges are aligned on 4^{J-1}, every rank owns the parents of all its cells and its pyramid is aligned: x_d, $m_{d,n}$ and their cotangents are sliced the same way at every level. With more ranks than cells at the coarsest levels, some ranks own no coarse cell; nothing else changes.

Each rank holds, at every level, its owned cells plus the halo cells that its blocks touch: the convolution domains of its blocks, the decimation domains of the blocks of the next level, the parents of its cells, and the supports of the decimation chains of its finer blocks (Sect. 2.2). These layouts are computed once. The neighbours are found with HEALPix neighbourhoods at the parent level, and each rank tells the owners of its halo cells which cells it needs; the counts on both sides are checked once at setup.

Two kinds of exchange cross rank boundaries during the sweeps (the ghost-cell pattern of domain decomposition; Smith et al. 1996): *halo exchanges*, where owners send the current values of the cells their neighbours hold (maps after each decimation, stored moduli before they are read, cotangents before the transposed decimations), and *ghost reductions*, where the contributions that a rank's pieces and transposed pieces wrote into cells owned by a neighbour are sent to the owner and added. Owners add the contributions of the other ranks in rank order, so the result does not depend on the arrival order of the messages. Both are point-to-point messages with neighbours only, one tag per exchange; the exchanges of all the stored moduli of a level are batched, so an evaluation needs about $5L$ exchanges with each neighbour (26 at $N_{\rm side} = 64$, about 56 at 2048, instead of 58 and 218 before batching). The partial sums of the coefficients are gathered and broadcast in rank order (one vector of N_c complex numbers per evaluation), so that the statistics, the loss and every decision of the optimiser are identical on every rank and do not depend on the number of ranks: 1, 2 and 4 ranks agree to 7×10^{-16}, and the $N_{\rm side} = 1024$ runs of Table 2 on 24 to 192 ranks and 1 to 4 nodes give the same losses to 16 digits.

No rank ever holds a full-sky map: the target and the initial map are read by owned slices, the white noise of the initial map is drawn per pixel from (seed, pixel) so that it does not depend on the decomposition, and the outputs are written by slices at their offsets.

3.2. Distributed L-BFGS

The L-BFGS history vectors are distributed like the map. The two-loop recursion and the strong Wolfe line search need only dot products and norms, which are global sums of per-rank partial sums (in rank order). The port of `torch.optim.LBFGS` goes through an abstract reduction, so the same code runs on one process, on thread ranks (for the tests), and under MPI.

3.3. Ensembles

An ensemble of B syntheses is optimised jointly with the loss on the mean of their statistics,

$$\mathcal{L}_S = \frac{1}{N_c}\sum_i \frac{|\bar S_i - S_i(t)|^2}{\sigma_i^2/B}, \qquad \bar S_i = \frac{1}{B}\sum_{m=1}^{B} S_i(x_m), \tag{7}$$

so that the ergodic scatter of the target goes into the differences between the members instead of being forced to zero in each of them. The mean alone leaves the scatter free (at $N_{\rm side} = 64$ every member of a 4-member ensemble ends 3–4.6σ from the target, where an ergodic realisation sits about 1σ away); a second-moment term holds it at the ergodic variance:

$$\mathcal{L}_D = \frac{w_D}{N_c}\sum_i \left(\frac{\mathcal{D}_i}{\sigma_i^2} - 1\right)^2, \qquad \mathcal{D}_i = \frac{1}{B-1}\sum_m |S_i(x_m) - \bar S_i|^2. \tag{8}$$

With $w_D = 1$ the members end at about 1σ from the target, with a scatter at the ergodic variance (0.94–1.12 per family at $N_{\rm side} = 64$).

The ranks are split into G groups of consecutive ranks (`MPI_Comm_split`); a group shares the sky of B/G members, which its engine evaluates as a batch, and the groups exchange one vector of N_c complex partial sums (and one of N_c reals for Eq. 8) per evaluation. $G = 1$ costs no extra communication and suits small maps; $G = B$ gives each member the memory and time of a single synthesis on P/B ranks or devices. Any G gives the same synthesis up to the order of the sums (10^{-12} relative after 30 iterations at $N_{\rm side} = 64$).

3.4. Overlap mode

An alternative decomposition (option `--overlap`, CPU engine) trades computation for communication. Every rank evaluates, for the fine levels ($d < n_c$), its owned blocks and the halo blocks they depend on, on extended regions, so that the sweeps need no exchange; the coarse levels ($d \geq n_c$), small, are held whole and evaluated by rank 0. An evaluation then needs only the level-0 map halo, one global sum of the coarse stored moduli and of the map of level n_c, the coefficient sum, one broadcast of the coarse cotangents and the level-0 ghost reduction of the map cotangent. It agrees with the standard mode to 7×10^{-16} on every fixture with 1, 2 and 4 ranks. [TODO: timings of the overlap mode at scale.]

3.5. Performance

Table 2 and Fig. 1 give the time of one evaluation (statistics and gradient) on three clusters: Datarmor (Ifremer; 2× Xeon E5-2680 v4 per node, Intel MPI), the hpc cluster (2× AMD EPYC 9655 per node, OpenMPI 5) and Irène Rome (TGCC; AMD EPYC 7H12, OpenMPI 4.1). At $N_{\rm side} = 64$ the time drops by 10 at 28 ranks on Datarmor and by 11.5 at 128 ranks on hpc, where it saturates (a rank owns 384 pixels of level 0 at 128 ranks). At $N_{\rm side} = 1024$ the efficiency is 75% from 24 to 192 ranks on one hpc node, and spreading the 192 ranks over 4 nodes costs 5%; on Irène it is 85% from 128 to 512 ranks. $L = 4$ costs 1.3 to 1.5 times $L = 2$ (12.8 s per evaluation on 512 Irène ranks at $N_{\rm side} = 1024$, 33.1 s on 160 ranks). Ensembles add the members' work and one exchange of partial sums between groups: a production ensemble of 10 maps at $N_{\rm side} = 1024$, $L = 4$, with all the terms of Sect. 5, runs at 128.4 s per evaluation (all members) on 640 Irène cores in 10 groups of 64 (316 evaluations in 11.3 h, the loss going from 5.7×10^4 to 2.46); the same ensemble without the κ terms ran at 149 s on 160 hpc cores in 10 groups of 16. The state per rank (maps, stored moduli, cotangents, L-BFGS history) shrinks as $1/P$: 123 MB at 24 ranks, 17 MB at 192 for $N_{\rm side} = 1024$, $L = 2$. In the Irène ensemble a rank owns 196 608 cells of level 0 and holds 221 760 with its halos; its state is 52.5 MB, of which 37.7 MB of L-BFGS history, 9.7 MB of stored moduli and cotangents and 5.1 MB of maps and cotangents (the halos weigh most on the stored moduli, which live on the coarse levels). What remains is the transients of one block (80–160 MB for blocks of 16 384 pixels, halved with blocks of 4096) and the MPI runtime (20–30 MB per rank for Intel MPI, 200–350 MB for OpenMPI 5). For comparison (on another machine), STL-Dev on the $N_{\rm side} = 64$ case takes 12.6 to 18 s per evaluation on 4 CPU threads with a peak of 1.9 GB, against 8.1 s and 103 MB for one STL-Dev-MPI rank on Datarmor.

Table 2. Seconds per evaluation on CPUs (kernel 5, blocks of 16 384 pixels, one map).

$N_{\rm side}$	L	Ranks	Machine	s / eval.
64	2	1	Datarmor	8.06
64	2	28	Datarmor	0.81
64	2	1	hpc	3.26
64	2	128	hpc	0.28
1024	2	24	hpc, 1 node	60.9
1024	2	192	hpc, 1 node	10.2
1024	2	4×48	hpc, 4 nodes	10.6
1024	2	128	Irène Rome	30.2
1024	2	512	Irène Rome	8.9
1024	4	512	Irène Rome	12.8
1024	4	160	[TODO: machine]	33.1

4. GPU engine

The GPU engine runs the same sweeps on one CUDA device per MPI rank, with the CPU decomposition, exchanges, objective and L-BFGS. Its kernels are compiled at start-up by NVRTC to a native binary for the device, with fused multiply-adds disabled and IEEE division and square root, and loaded through the `cudarc` crate (Lowman 2026), so the code builds without a CUDA toolkit. Every expression is evaluated in the order of its CPU counterpart, there are no floating-point atomics, and the results are identical from run to run; they differ from the CPU engine only through the transcendental functions and a few documented summation orders (statistics within 6×10^{-16}, level-0 gradient within 9×10^{-13} of its maximum). With one rank, nothing leaves the device; with several, the halo cells are exchanged through the host.

Blocks and geometry. On the device a level is processed by blocks of up to 2^{20} owned cells, sized from the free memory, hundreds of times larger than on the CPU; the layouts do not depend on the block size, so device blocks read the same halos. The convolution geometry (rotated stencils, ring bilinear interpolation, binding to the rank's grid) is computed by a kernel per output row and gauge, then transposed by grid cell so that the adjoint convolution is a gather in a fixed order. Within a memory budget, the geometry of the blocks is kept between sweeps either whole (24 bytes per entry) or as its transposition only (4 bytes per entry, the weights being recomputed).

Decimations. Instead of pieces, the device applies the full-sky decimation rows of the rank's owned coarse cells to the halo-exchanged finer field; the reverse chain is accumulated from the coarsest level by Horner's scheme. These are the values of the pieces, summed in another order.

One engine per rank. A synthesis evaluates several objectives on the same decomposition: the target statistics, the synthesis itself and, for weak lensing (Sect. 5), the statistics of $\kappa = f(g)$. One engine per rank is shared by all of them (topology, decimation rows, geometry cache and level fields), since an evaluation of one objective ends before the next starts. A plan without S_3 or S_4 neither computes nor allocates the moduli, the stored moduli and their chains. The level fields of the full plan then take $8BCH_0(8/3 + 56L/9)$ bytes for H_0 cells held at level 0 and B members, and the geometry cache takes most of what remains. Before this change, a 10-member ensemble at $N_{\rm side} = 1024$ with the κ term did not fit on one 141 GB H200 without capping the geometry caches; on a 6 GB laptop GPU, a 2-member ensemble at $N_{\rm side} = 128$ with the κ term, which needed such a cap, now runs 2.6 times faster per evaluation, with identical losses.

Performance (Table 3). On the Tesla T4, whose double-precision throughput is 1/32 of its single precision, three devices

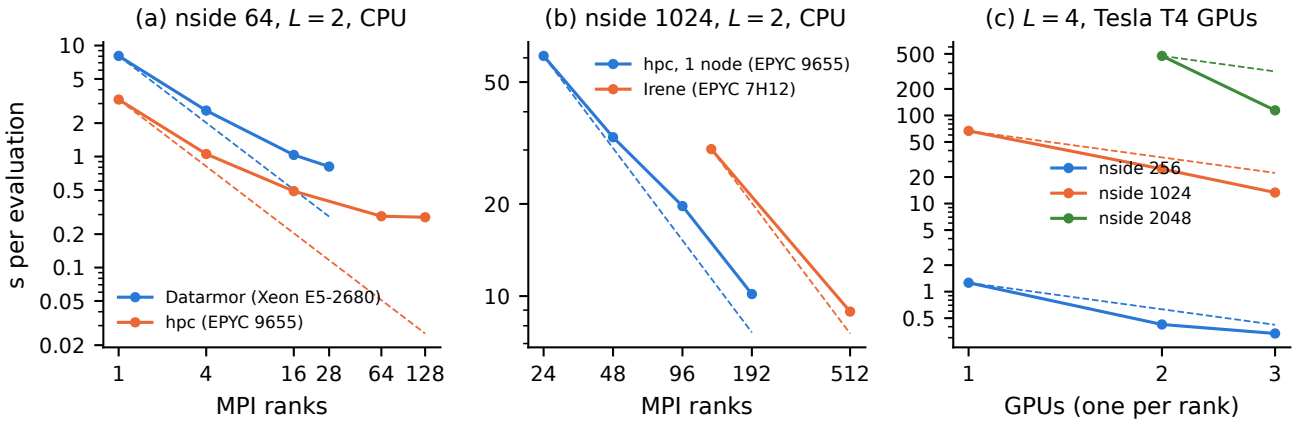


Fig. 1. Strong scaling of one evaluation (statistics and gradient). (a) $N_{\rm side} = 64$, $L = 2$, on Datarmor and hpc; (b) $N_{\rm side} = 1024$, $L = 2$, on one hpc node and on Irène Rome; (c) $L = 4$ on one to three Tesla T4 GPUs (garoupe) at $N_{\rm side} = 256$, 1024 and 2048 (one T4 cannot hold $N_{\rm side} = 2048$). Dashed lines: ideal scaling from the first point.

Table 3. Seconds per evaluation on GPUs ($L = 4$, kernel 5, double precision). B: maps synthesised together; "mem.": device memory peak per GPU, cache included.

$N_{\rm side}$	B	GPUs	s / eval.	mem. (GB)
256	1	1 T4	1.26	9.1
256	1	3 T4	0.34	4.0
1024	1	1 T4	66.8	12.3
1024	1	3 T4	13.3	10.9
2048	1	2 T4	474	13.8
2048	1	3 T4	114	12.9
512	10^a	2 T4	50.6	13.3
1024	10^a	1 H200	54.3	98
2048	5^a	1 H200	157	139
4096^b	1	1 H200	~325	–
4096^b	1	2 H200	67.9	124
4096^b	1	16 V100	25.2	13.9

Notes. [a] Ensembles with the κ terms of Sect. 5. [b] Synthesis of the $N_{\rm side} = 2048$ convergence map at twice its resolution ($J = 11$, 19 450 coefficients); the H200 memory is mostly geometry cache, which fills the device. T4: garoupe; H200: hpc cluster; V100: Irène (4 nodes of 4 V100 16 GB, job 6048940).

take 13.3 s per evaluation at $N_{\rm side} = 1024$, $L = 4$, and 114 s at 2048, where the geometry no longer fits and is rebuilt at each sweep; three T4s are twice as fast as a 192-core EPYC node at $N_{\rm side} = 1024$, $L = 2$. The scaling from 1 to 3 T4s is superlinear while the geometry cache covers more of each rank's rows. On H200 GPUs, an ensemble of 10 syntheses at $N_{\rm side} = 1024$ runs on a single device, and a single map at $N_{\rm side} = 4096$ (2×10^8 pixels, $J = 11$) takes about 325 s per evaluation on one device and 67.9 s on two, a superlinear gain as on the T4s, where it comes from the geometry cache covering more of each rank's rows. The same $N_{\rm side} = 4096$ synthesis runs on 16 V100s with 16 GB each (25.2 s per evaluation, 13.9 GB per device): small devices are usable as long as there are enough of them. On Irène, a node of 4 V100s is worth about 1024 AMD Rome cores (23.4 against 28.8 s per evaluation for the same synthesis at $N_{\rm side} = 2048$).

5. Synthesising the weak-lensing convergence

5.1. Gaussianised synthesis

The convergence κ at low source redshift is dominated by rare, extended haloes: the map used below has a skewness of 8.4 at $N_{\rm side} = 64$. Synthesising κ directly works poorly, and we synthesise its Gaussian anamorphosis instead, as in FOSCAT (Delouis 2022): the pixel of rank r among N becomes $g = \Phi^{-1}((r + 1/2)/N)$, and a synthesis g is mapped back by matching the target's histogram. Three choices made the synthesis of g reliable:

- $L = 4$ *orientations.* With $L = 2$ the syntheses are elongated east–west (east–west to north–south small-scale power 0.89±0.03 against 1.02±0.05 for the target at $N_{\rm side} = 256$).
- *Coarse to fine.* Each resolution starts from the previous one replicated on the children, plus white noise with the rms of the target's scales missing at the coarser resolution; ensembles carry each member to the next resolution.
- *Power of every pyramid level.* The scattering coefficients leave the finest scale loose: with every family below 0.5σ, a synthesis at $N_{\rm side} = 256$ holds only 58% of the variance of the finest details. Writing d_n for the details of level n (the map at $N_{\rm side} = n$ minus the mean of each group of four children) and V_n for their variance, we add

$$\mathcal{L}_P = \frac{w_P}{B} \sum_m \sum_n \left(\frac{V_n(g_m)}{V_n(t)} - 1 \right)^2 , \qquad (9)$$

with its exact gradient; at $w_P = 1$ every level is within 0.3% of the target, at no cost and with the statistics unchanged.

5.2. A Gaussian copula at small scales

Ensembles synthesised this way have every family of the gaussianised statistics at about 1σ per member and every level variance within 4%, and yet their κ power spectrum is 15–20% low at $\ell = 135$–284 and 26% high at $\ell = 600$–767 at $N_{\rm side} = 256$, the same for every member. A Gaussian random field with exactly the target's spectrum of g, mapped through the same table, has the same deficit above $\ell = 100$: at small scales the syntheses have a Gaussian copula. The κ power there is carried by the extreme tail of g (25 pixels above 4σ and 1062 above 3σ out

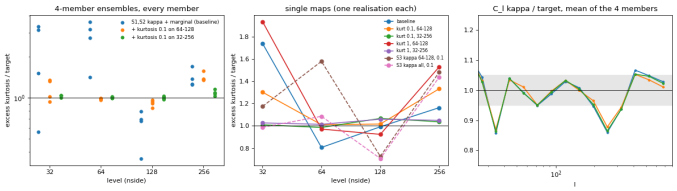


Fig. 2. Kurtosis of the κ details per level over the target's, at $N_{\rm side} = 256$: 4-member ensembles (left) and single maps (middle) with the κ terms, without and with the kurtosis term on two level ranges; right: κ power spectrum over the target's. [TODO: redraw in the paper's style.]

of 786 432), whose arrangement in extended haloes weighs almost nothing in the gaussianised statistics. Synthesising a less gaussianised field (logarithm or κ itself) removes the deficit but loses the bulk (8 to 107σ off in gaussianised space); correcting the spectrum afterwards breaks the finest level; and the power of each level of κ is met by isolated spikes far in the tail.

5.3. Terms on $\kappa = f(g)$

Three terms, evaluated on $f(g)$ with f the monotone cubic interpolation (Fritsch & Carlson 1980) of the gaussianisation table through 256 knots (linear beyond the ends), fix this:

- the reduced χ^2 of the mean, variance, S_1 and S_2 of κ, as Eq. (7) with its own plan, target and ergodic weights, with a factor $w_\kappa = 0.1$;
- the squared 1-D Wasserstein distance (Villani 2009) between the sorted values of each member and of the target,

$$\mathcal{L}_W = \frac{w_W}{B} \sum_m \frac{1}{N} \sum_p \left(g_{m,(p)} - t_{(p)}\right)^2, \tag{10}$$

  computed by a distributed sample sort (Frazer & McKellar 1970). Without it the κ term is met by pushing values of g off the Gaussian marginal, which the final histogram matching undoes; with it, $f(g)$ is the map the matching produces;
- the kurtosis of the details of κ at the levels $32 \le n \le N_{\rm side}$,

$$\mathcal{L}_K = \frac{w_K}{B} \sum_m \sum_n \left(\frac{K_n(f(g_m))}{K_n(\kappa_t)} - 1\right)^2, \qquad K_n = \frac{\langle d_n^4 \rangle}{\langle d_n^2 \rangle^2}, \tag{11}$$

  since the first two terms make the tails of the details too heavy at intermediate scales (excess kurtosis 3 times the target's at $N_{\rm side} = 64$ in a 4-member ensemble at $N_{\rm side} = 256$).

All gradients are exact (checked against finite differences) and chained through $f'(g)$. The recommended weights for B members are $w_\kappa = 0.1$, $w_W = 1000B$, $w_P = B$, $w_K = 0.1B$ and $w_D = 1$: the terms on the members are sums over the members divided by B, so their weights per member are 1000, 1 and 0.1.

Table 4 summarises the effect at $N_{\rm side} = 256$ (4-member ensembles started from $N_{\rm side} = 128$ members, 200 iterations): the κ terms bring the spectrum from 12.5% to 6.6% rms over $\ell = 31$–767, every family of g staying at 1σ per member; the kurtosis term brings every level within 8% of the target's kurtosis without moving anything else. The κ term costs 1.8 times the evaluation (one more set of wavelet transforms, without S_3/S_4), the kurtosis term 3%.

Table 4. Effect of the κ terms on 4-member ensembles at $N_{\rm side} = 256$ (from `docs/kappa_power_spectrum.md`).

	none	$+\kappa$, W_2	+ kurtosis
$C_\ell(\kappa)$, rms (max)	12.5% (24%)	6.6% (14%)	6.5% (14%)
S_1–S_4 of g (σ)	0.91–1.00	0.90–1.03	0.88–1.04
Kurtosis, $n = 32, 64$	0.6, 1.1	2.3, 3.0	1.03, 1.01
Kurtosis, $n = 128, 256$	–	0.68, 1.32	1.00, 1.08
Cost / evaluation	1	1.8	1.85

Notes. Columns: no κ term, then the κ terms with the Wasserstein distance, then with the kurtosis term. $C_\ell(\kappa)$: rms and maximum relative deviation of the ensemble's κ spectrum from the target's over $31 \le \ell \le 767$; S_1–S_4: residuals per member in units of σ. Kurtosis ratios: medians over the members of the excess kurtosis of the κ details over the target's (50, 100, 340, 513 at $n = 32, 64, 128, 256$); for the control ensemble, 30 and 114 at $n = 32$ and 64 (not measured at 128 and 256). The kurtosis ratios of the second and third columns come from a rerun of the second on a T4.

6. Ensembles of syntheses of the Takahashi convergence

6.1. Data and protocol

The target is the convergence map r000 of the full-sky ray-tracing simulations of Takahashi et al. (2017) for the first source plane ($z_s \simeq 0.05$, `zs1`), given at $N_{\rm side} = 4096$ and degraded to NESTED maps at $N_{\rm side} = 256$ to 2048 by averaging sub-pixels. The nine realisations r001 to r009 of the same suite, with independent initial conditions, measure the cosmic variance of every statistic: none of them is used in the synthesis. An ensemble *matches* the target when its members deviate from r000 by about as much as the realisations deviate from each other.

The ensembles are built from coarse to fine ($N_{\rm side} = 128$, 256, 512, 1024, then 2048), each resolution started from the members of the previous one, with the loss of Eqs. (7), (8) and the terms of Sect. 5, $L = 4$, $K = 5$, $J = \log_2 N_{\rm side} - 1$, and a few hundred L-BFGS iterations per step. Table 5 lists the runs. Each member's g is mapped back to κ by histogram matching with r000.

6.2. Cosmic variance

Over the ten realisations, the relative standard deviation is 8–18% for the variance of the κ details per level, 13–35% for their excess kurtosis, 7–17% for the power spectrum in logarithmic bands above $\ell = 30$, 6–9% for the peak counts above 3, 5 and 10σ, and only 1–4% for the variance of the details in gaussianised space: the rank transform removes the halo-dominated tail, and what remains is a much more precise statistic. r000 is a typical realisation, within 1.9σ of the mean of the ten for every statistic (Table 6).

6.3. Results

Figures 3 and 4 show the target, another realisation and two members; Figs. 5–8 and Table 6 compare the statistics. We quote the $N_{\rm side} = 1024$ ensembles below.

Power spectrum. Without the κ terms the members' spectrum is 24% rms off r000 over $\ell \ge 31$, up to 8σ in the last band. With them, it is within 5.9% rms and within 1σ of r000 in every band (median $|z| = 0.24$; Fig. 5). At $N_{\rm side} = 512$ both ensembles, which carry the κ terms, are within 3.4–4.4% rms and 0.6σ.

Table 5. Ensembles of 10 syntheses used in Sect. 6.

$N_{\rm side}$	Label	Loss terms	Iterations	s / eval.	Hardware
512	no kurtosis	S, D, P, κ, W	300	50.6	2 T4 (2 groups)
512	kurtosis	S, D, P, κ, W, K	300 + 150	41.3	2 T4 (2 groups)
1024	no κ terms	S, D, P ($w_P = 1$)	300	149	160 EPYC 9655 cores (10 groups)
1024	κ terms	S, D, P, κ, W	300	54.3	1 H200
2048	[TODO: Irène, job 6090003]	[TODO:]	[TODO:]	[TODO:]	[TODO: Irène Rome CPUs]
2048	[TODO: hpc, job 2059308, 5 maps]	[TODO:]	[TODO:]	[TODO:]	[TODO: 1 H200]

Notes. S: scattering covariance of the ensemble mean (Eq. 7); D: scatter (Eq. 8, $w_D = 1$); P: power per level ($w_P = 10$ unless stated); κ: mean, variance, S_1, S_2 of κ ($w_\kappa = 0.1$); W: Wasserstein ($w_W = 10^4$); K: kurtosis ($w_K = 1$, levels 32 to $N_{\rm side}$). [TODO: 2048 settings from the production chain.]

Per-level statistics (Fig. 6). The variance of the κ details is within 1.1σ of r000 at every level with the κ terms, against $+2.5\sigma$ and -2.1σ at the levels 256 and 1024 without. The excess kurtosis overshoots at intermediate levels with the κ terms and no kurtosis term ($+10.7\sigma$ in median at level 64); at $N_{\rm side} = 512$ the kurtosis term brings every level from 32 to 256 within 1σ (level 64 from +5.3 to 0.0σ), the finest level remaining at $+2.3\sigma$. The variance in gaussianised space, the most precise statistic, is where the ensembles are furthest from the target: +12 and $+14\sigma$ (+16 and +22%) at the levels 128 and 256, and -8.7σ (−31%) at the finest level for $N_{\rm side} = 1024$.

Peaks and PDF (Fig. 7). The counts of peaks above 3, 5 and 10σ are within 1σ with the κ terms, against $+4\sigma$ (above 3σ) without. The histograms of peak heights at $N_{\rm side}/4$ show the κ-term ensemble with 20–30% too many peaks between 3 and 10σ at that resolution.

Minkowski functionals (Fig. 8, Appendix A) are the most demanding test: none of the terms of the loss constrains them directly. The members deviate from r000 by 2.1–2.5σ rms over the thresholds $-2 \le \nu \le 5$ for the map smoothed over 4 pixels (13.7 arcmin), and by 1.1–2.5σ for a 30 arcmin smoothing, with the largest deviations at $\nu \simeq 1$–3 for V_1 and V_2, i.e. in the connectivity of the moderately high excursion sets. Per pyramid level (Table 7 and Fig. 9: maps degraded by 4-children means, no smoothing), the reference is the deviation of the realizations r001–r009 from r000, a median $|z|$ of 0.8 to 2.0 and a maximum of 3.2 to 3.8 (r000 itself sits 1.5–2σ from the mean of the realizations in V_0 at the levels 128 to 512). At level 64 the members' median $|z|$ (0.9–1.5) is that of the realizations, apart from a few thresholds in the low tail. At the levels 256 and 512 it is 3.0–3.9, with systematic offsets of 3–7σ in V_1 and V_2 for $0 \lesssim \nu \lesssim 3$, the largest deviations being at $\nu \lesssim 0$ (the troughs). At the native level the members are tens to hundreds of σ off, the trace of the deficit of the finest gaussianised level.

Table 6. Deviation of the ensemble members from the target r000 at nside 1024, in units of the standard deviation σ of the 10 Takahashi realizations: median [min, max] over the members (bold: $|z| > 2$). σ/μ is the realizations' relative standard deviation and the r000 column the target's own deviation from their mean. Details of level n: the map at nside n minus the mean of each group of 4 children. The C_ℓ and Minkowski rows give the rms over the bands ($\ell \geq 31$) or thresholds ($-2 \leq \nu \leq 5$, FWHM 13.74′) of each member's deviation.

Statistic	Level	σ/μ	r000	no kappa terms	kappa terms
variance, κ					
	8	13.4%	+0.1	−0.7 [−1.1, +1.8]	−0.3 [−0.8, +0.6]
	16	18.0%	−0.2	−0.2 [−0.4, +1.8]	+0.2 [−0.3, +0.6]
	32	15.0%	−0.1	−0.5 [−1.0, +1.4]	−0.0 [−0.6, +0.9]
	64	14.7%	−0.1	−0.2 [−0.4, +0.4]	+0.4 [−0.2, +0.9]
	128	13.0%	+0.2	+0.7 [+0.2, +1.6]	+0.0 [−0.5, +1.1]
	256	11.7%	+0.1	**+2.5** [**+2.2**, **+2.7**]	−0.0 [−0.5, +0.2]
	512	10.9%	−0.1	−0.9 [−1.9, −0.5]	−0.2 [−0.4, −0.0]
	1024	9.3%	−0.0	**−2.1** [**−3.3**, −1.7]	−0.1 [−0.6, +0.4]
excess kurtosis, κ					
	8	31.4%	+0.7	**−2.3** [**−3.0**, **+32.8**]	−1.3 [−1.7, **+3.4**]
	16	35.2%	+1.4	−1.7 [**−2.3**, **+61.8**]	**+2.7** [−1.7, **+5.5**]
	32	35.3%	+1.1	**−2.0** [**−2.7**, **+23.1**]	**+2.9** [−1.0, **+10.8**]
	64	27.9%	+0.7	+0.0 [−1.7, **+6.5**]	**+10.7** [**+4.5**, **+35.3**]
	128	29.8%	+1.9	−0.8 [**−2.3**, **+11.7**]	**+3.3** [−1.5, **+10.7**]
	256	21.3%	+1.6	**+4.8** [**+2.5**, **+9.6**]	+1.7 [−1.9, **+4.8**]
	512	16.3%	+1.4	**+2.8** [−1.4, **+6.5**]	−0.0 [−1.5, **+2.6**]
	1024	17.3%	+1.1	**+2.7** [**−2.4**, **+5.7**]	**+3.6** [+1.2, **+10.6**]
variance, gaussianised					
	8	5.6%	+0.6	−0.2 [−0.6, +0.3]	−0.1 [−0.3, +0.2]
	16	3.7%	−0.7	−0.1 [−0.5, +0.5]	+0.1 [−0.1, +0.4]
	32	2.2%	−1.1	+0.6 [+0.2, +1.1]	+1.2 [+0.9, +1.6]
	64	2.0%	−1.3	**+3.0** [**+2.9**, **+3.4**]	**+4.0** [**+3.9**, **+4.3**]
	128	1.3%	+0.2	**+12.6** [**+12.2**, **+12.7**]	**+12.0** [**+11.8**, **+12.2**]
	256	1.6%	+1.1	**+22.4** [**+22.2**, **+22.6**]	**+13.9** [**+13.7**, **+14.1**]
	512	2.4%	+1.3	+0.2 [+0.1, +0.4]	+0.7 [+0.6, +0.9]
	1024	3.6%	+1.1	**−11.1** [**−11.1**, **−11.0**]	**−8.7** [**−8.8**, **−8.5**]
peaks					
	$> 3\sigma$	5.8%	−0.4	**+4.0** [**+3.8**, **+4.1**]	−0.2 [−0.4, −0.1]
	$> 5\sigma$	6.6%	−0.3	**+3.1** [**+2.8**, **+3.3**]	+0.3 [+0.1, +0.5]
	$> 10\sigma$	8.1%	−0.2	+1.7 [+1.3, **+2.1**]	+0.8 [+0.6, +1.1]
C_ℓ					
	rms, $\ell \geq 31$			2.6 [2.3, 2.8]	0.6 [0.5, 1.0]
Minkowski					
	V_0, rms			1.8 [1.4, 3.2]	2.3 [2.0, 2.4]
	V_1, rms			1.3 [1.1, 3.7]	2.2 [1.8, 2.7]
	V_2, rms			3.5 [3.0, 5.4]	2.5 [2.4, 2.7]

Table 7. Minkowski functionals of κ degraded to each level (nside n, means of 4 children, no smoothing), thresholds $-2 \leq \nu \leq 5$: deviation from r000 in units of the standard deviation of the 10 realizations. Each cell: median $|z|$ and maximum $|z|$ over the members and the thresholds, and rms over the thresholds of the members' median z. The first column applies the same measure to the realizations r001–r009, the reference for an ensemble within cosmic variance.

Level		r001–r009	kappa terms	no kappa terms
64	V_0	1.0 / 3.4 / 1.1	1.5 / 14.6 / 3.0	1.8 / 10.8 / 1.9
64	V_1	1.1 / 3.4 / 1.1	1.1 / 15.3 / 1.2	1.4 / 9.0 / 1.6
64	V_2	1.1 / 3.5 / 1.1	0.9 / 14.7 / 1.2	1.1 / 9.5 / 1.5
128	V_0	1.6 / 3.6 / 1.7	2.2 / 22.0 / 3.1	2.4 / 26.2 / 5.7
128	V_1	1.2 / 3.6 / 1.2	1.9 / 14.1 / 2.2	2.7 / 16.9 / 4.3
128	V_2	1.0 / 3.6 / 1.1	1.6 / 11.2 / 2.0	2.0 / 11.2 / 2.9
256	V_0	2.0 / 3.6 / 2.0	3.0 / 11.0 / 3.6	1.6 / 76.3 / 21.5
256	V_1	1.2 / 3.8 / 1.3	3.0 / 9.8 / 3.1	2.7 / 54.9 / 14.4
256	V_2	1.2 / 3.7 / 1.2	3.4 / 13.2 / 3.7	2.6 / 38.6 / 7.6
512	V_0	2.0 / 3.6 / 2.0	3.1 / 35.1 / 7.1	1.4 / 353.3 / 75.0
512	V_1	1.1 / 3.7 / 1.2	3.9 / 21.5 / 5.3	2.0 / 157.8 / 35.8
512	V_2	0.9 / 3.4 / 0.9	3.5 / 24.3 / 5.9	1.5 / 90.3 / 21.3
1024	V_0	1.7 / 3.5 / 1.8	0.5 / 571.1 / 67.4	1.8 / 1235.2 / 316.8
1024	V_1	1.1 / 3.5 / 1.1	4.2 / 1174.1 / 89.1	7.7 / 3489.1 / 513.1
1024	V_2	0.8 / 3.2 / 0.8	3.1 / 129.0 / 26.3	2.4 / 578.2 / 133.5

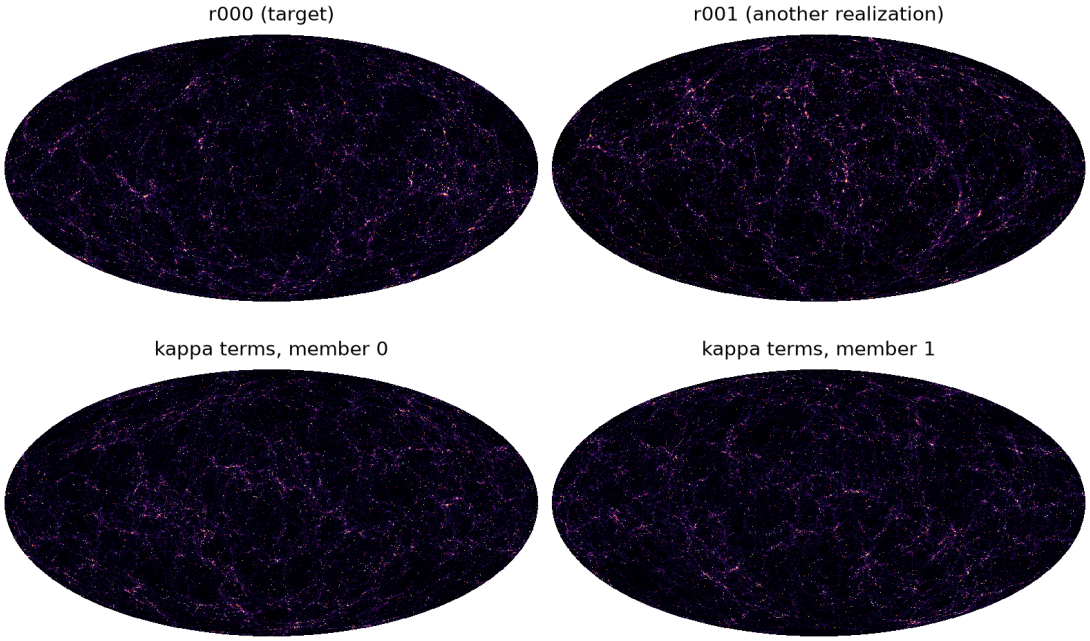


Fig. 3. Convergence maps at $N_{\rm side} = 1024$ (Mollweide): the target r000, an independent realisation r001 of the same simulation suite, and two members of the ensemble with the κ terms. Colour: κ/σ(r000), same range in every panel.

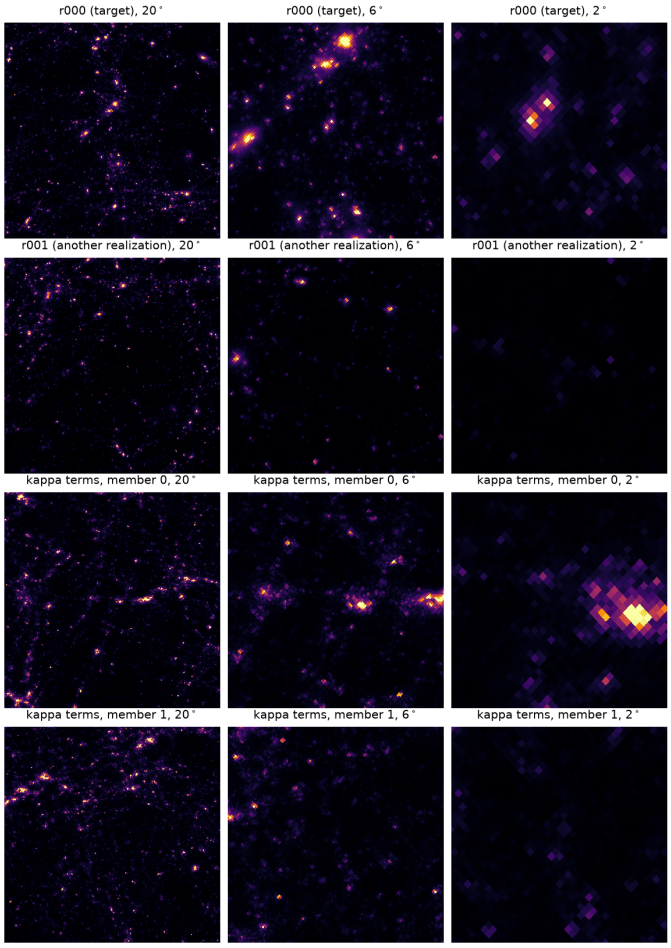


Fig. 4. Gnomonic zooms of 20, 6 and 2 degrees on the same centre in the maps of Fig. 3 (rows: r000, r001, two members), same colour scale.

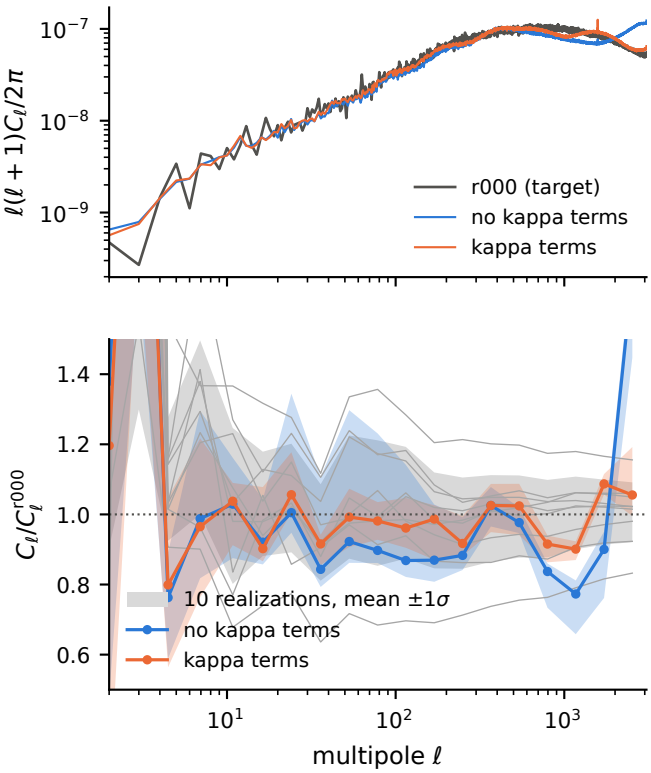


Fig. 5. Power spectrum of κ at $N_{\rm side} = 1024$. Top: $\ell(\ell+1)C_\ell/2\pi$ of r000 and of the members (median). Bottom: ratio to r000 in logarithmic bands; grey: mean $\pm1\sigma$ of the 10 realisations, thin lines r001–r009; colours: median and range of the members.

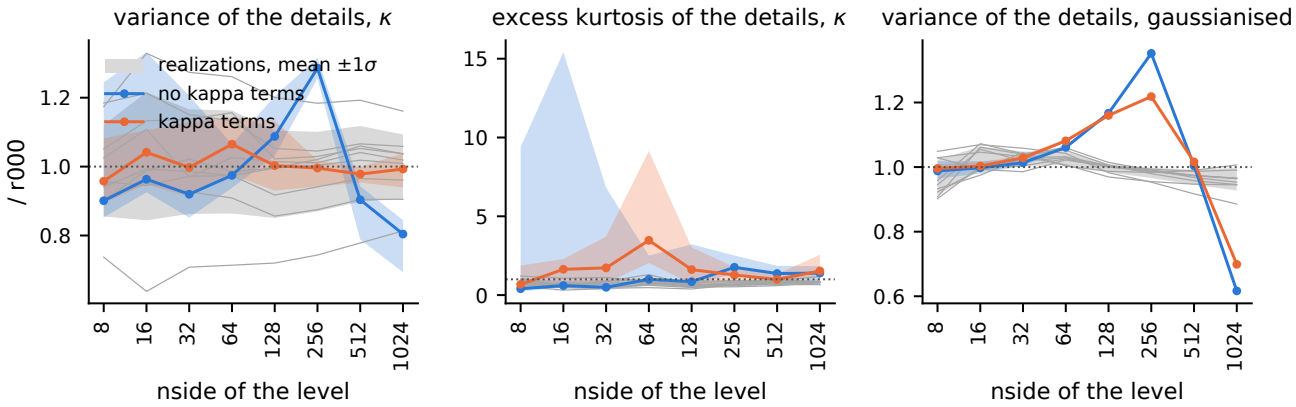


Fig. 6. Statistics of the 4-children pyramid details per level, over r000's: variance and excess kurtosis of κ, variance in gaussianised space (each map's own rank transform). Grey: realisations, mean $\pm 1\sigma$ and individual curves; colours: median and range of the members.

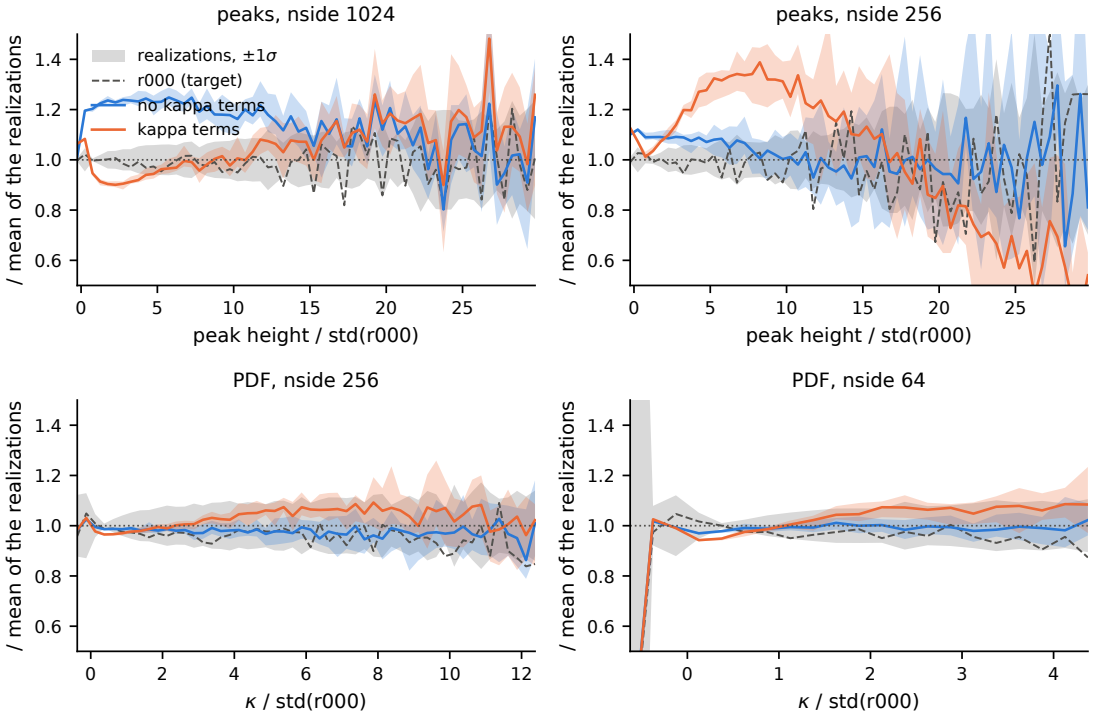


Fig. 7. Top: number of peaks (pixels higher than their 8 neighbours) per bin of height, at $N_{\rm side}$ and $N_{\rm side}/4$; bottom: one-point PDF of κ degraded to $N_{\rm side}/4$ and $N_{\rm side}/16$ (at full resolution the members have r000's histogram by construction). All divided by the mean of the 10 realisations.

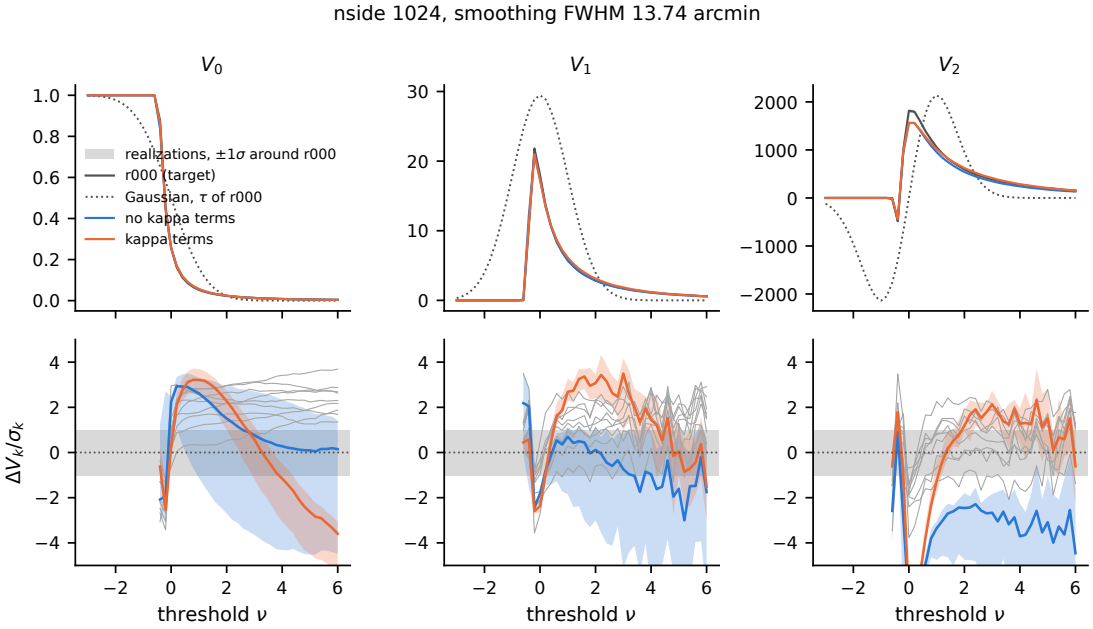


Fig. 8. Minkowski functionals of κ smoothed over 4 pixels. Top: V_0, V_1, V_2 of r000, of the members (median) and the Gaussian expectation for r000's spectral moment; bottom: deviation from r000 in units of the realisations' standard deviation. The 30 arcmin smoothing is in `figures/minkowski_n1024_s1.pdf`.

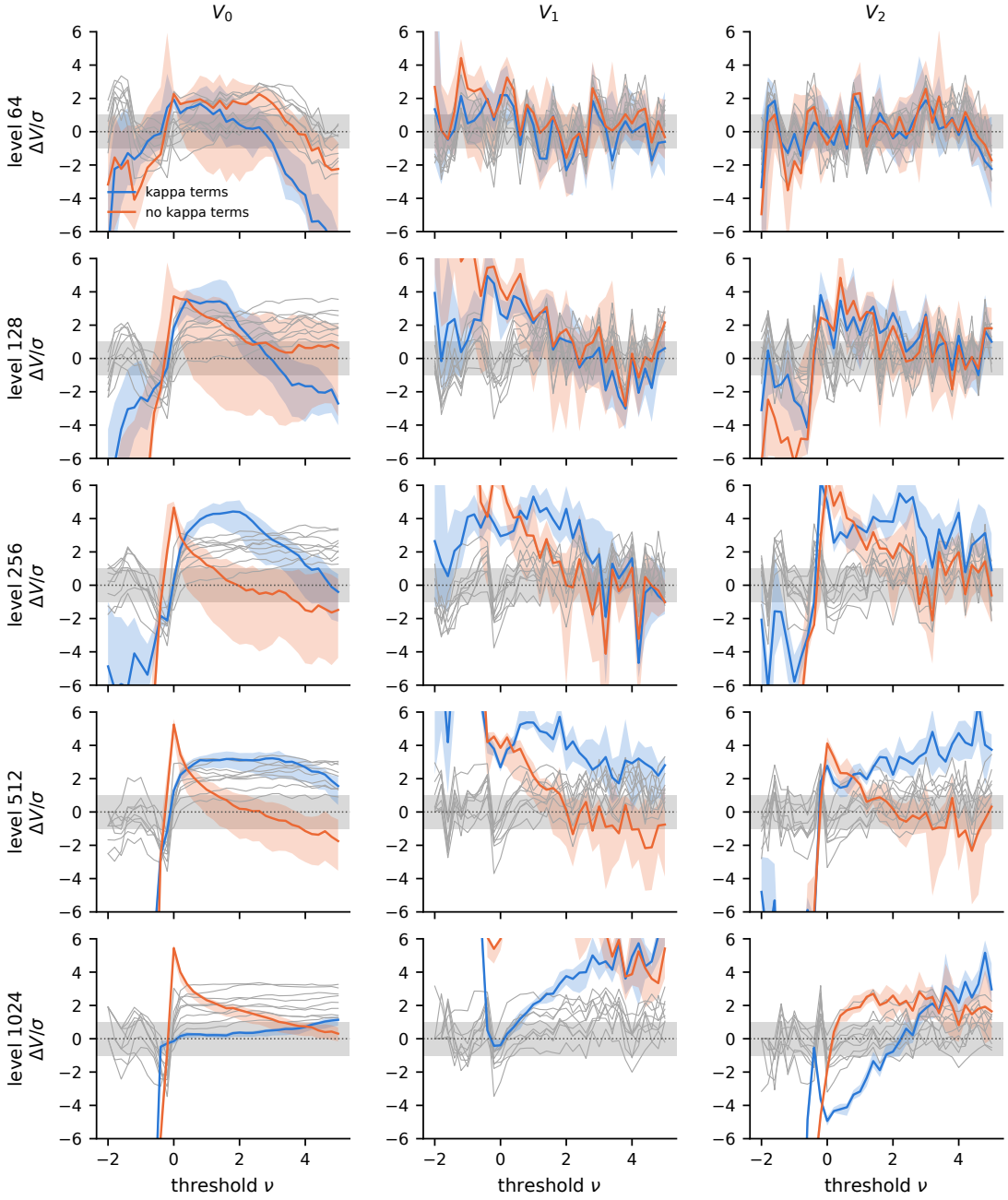


Fig. 9. Minkowski functionals of κ degraded to each level of the pyramid (means of 4 children, no smoothing): deviation from r000 in units of the standard deviation of the 10 realizations. Grey: r001–r009; colours: median and range of the members.

7. Discussion and outlook

7.1. What the distributed engine changes

The explicit, block-wise gradient turns the memory of a scattering-covariance synthesis from a property of the device into a property of the machine. The state of a synthesis is a few hundred bytes per pixel and per map, divided by the number of processes, and the transient memory is set by the block size, not by the map. On CPUs the time per evaluation drops with the number of ranks as long as a rank owns at least about 10^4 pixels of level 0; on GPUs the limit is the device memory, and several devices per synthesis lift it. An ensemble of ten $N_{\rm side} = 2048$ maps, 5×10^8 unknowns, is a routine job on a few hundred CPU cores or a few GPUs. The determinism of the implementation (identical losses for any rank count) makes such runs reproducible and comparable across machines.

7.2. Limits of the present syntheses

The weak-lensing ensembles show where the statistical model, rather than the optimiser, is the limit. The gaussianised level variances are 10–30% off at a few levels, far outside a cosmic variance of 1–4%; the spectrum of g oscillates from one octave to the next by about 10%, because dyadic wavelet bands fix the power of each octave but not its shape within it; and the Minkowski functionals, which probe the topology of the excursion sets, deviate by 2–3σ. The per-level kurtosis, held exactly by the kurtosis term, keeps no scatter between the members. These point to statistics to add (finer frequency sampling, cross-level terms on κ) rather than to more iterations. [TODO: update with the 2048 ensembles.]

7.3. Towards climate and ocean data

The fields of Earth-system science are larger than the cosmological maps of this paper in every direction: higher resolution (global kilometre-scale models and reanalyses), many channels (temperature, salinity, velocities, sea-surface height, at several depths or pressure levels) and time. The engine already handles C channels and their cross-covariances, and its memory per process is set by the share of the sky, so the first two directions are a matter of node count. A global ocean model at 1/12° has about 10^7 surface cells, comparable to $N_{\rm side} = 1024$; a kilometre-scale atmosphere on HEALPix at $N_{\rm side} = 8192$ to 16 384 has 0.8 to 3.2×10^9 cells per level, which at about 400 bytes per pixel and map (engine and L-BFGS, $L = 4$) needs 3 to 13 TB of state for an ensemble of ten: the memory of 5 to 20 nodes of the hpc cluster used here. Time is the larger cost: extrapolating the Irène rate linearly in the number of pixels (12.8 s per evaluation for 1.3×10^7 pixels on 512 cores, $L = 4$), one evaluation of a single $N_{\rm side} = 8192$ map would take about 14 minutes on 512 cores, or about a minute on 7×10^3 cores. Three extensions are needed: masks and missing data (land for the ocean, survey footprints in cosmology), which the block decomposition accommodates by dropping blocks and weighting halos; time, as an extra dimension of the wavelets or as cross-covariances between consecutive fields, which multiplies the channels but not the halos; and lower-precision arithmetic on GPUs, which would require validating the statistics in mixed precision. Such emulators would complement the large probabilistic forecasting models (Price et al. 2025): they need no training archive, they produce realisations constrained by interpretable statistics, and they extend to the resolution of the latest models at the cost of more nodes, not of a new network.

8. Conclusions

We have presented STL-Dev-MPI, a distributed implementation of scattering-covariance synthesis on HEALPix. Its explicit pyramidal, block-wise gradient bounds the memory of each process by its share of the sky; its MPI decomposition, with point-to-point halos, ordered ghost reductions and a distributed L-BFGS, scales with 75–85% efficiency to hundreds of cores and gives identical results for any rank count; its GPU engine runs ten $N_{\rm side} = 1024$ syntheses on one H200 and an $N_{\rm side} = 4096$ synthesis on two. Applied to the weak-lensing convergence, with terms on the statistics of κ, on the pixel histogram, on the power and kurtosis of each pyramid level and on the scatter of the ensemble, it produces ensembles whose power spectrum, per-level variance and peak counts lie within the cosmic variance of independent realisations, while the gaussianised level variances and

the Minkowski functionals show what the statistical model still misses. [TODO: conclusions of the 2048 ensembles.] The code, the fixtures and the scripts of every figure are in the STL-Dev-MPI repository, to be made public with the paper [TODO: URL / Zenodo DOI].

Acknowledgements. [TODO: Acknowledgements: computing centres (Ifremer Datarmor, the hpc cluster, TGCC Irène under projects gen17863 and gen17889), funding.] This work used HEALPix (Górski et al. 2005), healpy (Zonca et al. 2019), NumPy (Harris et al. 2020), SciPy (Virtanen et al. 2020), Matplotlib (Hunter 2007), rsmpi (Steinbusch et al. 2026), cudarc (Lowman 2026) and cdshealpix (Pineau 2026); the reference implementation is STL-Dev (STL-Proj 2025), with the anamorphosis of FOSCAT (Delouis 2022). MPI is the standard of Message Passing Interface Forum (2023).

References

Allys, E., Levrier, F., Zhang, S., et al. 2019, A&A, 629, A115
Allys, E., Marchand, T., Cardoso, J.-F., et al. 2020, Phys. Rev. D, 102, 103506
Bi, K., Xie, L., Zhang, H., et al. 2023, Nature, 619, 533
Bruna, J. & Mallat, S. 2013, IEEE Transactions on Pattern Analysis and Machine Intelligence, 35, 1872
Bruna, J. & Mallat, S. 2018, Mathematical Statistics and Learning, 1, 257
Campeti, P., Delouis, J.-M., Pagano, L., et al. 2025, A&A, 700, A136
Cheng, S. & Ménard, B. 2021, arXiv e-prints, arXiv:2112.01288
Cheng, S., Morel, R., Allys, E., Ménard, B., & Mallat, S. 2024, PNAS Nexus, 3, pgae103
Cheng, S., Ting, Y.-S., Ménard, B., & Bruna, J. 2020, MNRAS, 499, 5902
Delouis, J.-M. 2022, FOSCAT: generate synthetic Healpix or 2D data using Cross Scattering Transform, `https://github.com/jmdelouis/FOSCAT`, python package `foscat` on PyPI (`https://pypi.org/project/foscat/`), first released in 2022 (version 0.1); latest release 2026.6.1
Delouis, J.-M., Allys, E., Gauvrit, E., & Boulanger, F. 2022, A&A, 668, A122
Euclid Collaboration: Ajani, V., Baldi, M., Barthelemy, A., et al. 2023, A&A, 675, A120
Eyring, V., Bony, S., Meehl, G. A., et al. 2016, Geoscientific Model Development, 9, 1937
Frazer, W. D. & McKellar, A. C. 1970, Journal of the ACM, 17, 496
Fritsch, F. N. & Carlson, R. E. 1980, SIAM Journal on Numerical Analysis, 17, 238
Gatti, M., Jeffrey, N., Whiteway, L., et al. 2024, Phys. Rev. D, 109, 063534
Gatys, L. A., Ecker, A. S., & Bethge, M. 2015, in Advances in Neural Information Processing Systems, ed. C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, & R. Garnett, Vol. 28 (Curran Associates, Inc.)
Górski, K. M., Hivon, E., Banday, A. J., et al. 2005, ApJ, 622, 759
Harris, C. R., Millman, K. J., van der Walt, S. J., et al. 2020, Nature, 585, 357
Hersbach, H., Bell, B., Berrisford, P., et al. 2020, Quarterly Journal of the Royal Meteorological Society, 146, 1999
Hewitt, H. T., Roberts, M., Mathiot, P., et al. 2020, Current Climate Change Reports, 6, 137
Hunter, J. D. 2007, Computing in Science & Engineering, 9, 90
Jeffrey, N., Boulanger, F., Wandelt, B. D., et al. 2022, MNRAS, 510, L1
Karlbauer, M., Cresswell-Clay, N., Durran, D. R., et al. 2024, Journal of Advances in Modeling Earth Systems, 16, e2023MS004021
Kochkov, D., Yuval, J., Langmore, I., et al. 2024, Nature, 632, 1060
Kratochvil, J. M., Haiman, Z., & May, M. 2010, Phys. Rev. D, 81, 043519
Kratochvil, J. M., Lim, E. A., Wang, S., et al. 2012, Phys. Rev. D, 85, 103513
Lam, R., Sanchez-Gonzalez, A., Willson, M., et al. 2023, Science, 382, 1416
Liu, D. C. & Nocedal, J. 1989, Mathematical Programming, 45, 503
Lowman, C. 2026, cudarc: safe and minimal CUDA bindings, `https://github.com/chelsea0x3b/cudarc`, rust crate `cudarc`, version 0.19.10 (`https://crates.io/crates/cudarc`); repository formerly at github.com/coreylowman/cudarc
Mallat, S. 2012, Communications on Pure and Applied Mathematics, 65, 1331
McEwen, J. D., Wallis, C. G. R., & Mavor-Parker, A. N. 2022, in International Conference on Learning Representations (ICLR 2022)
Message Passing Interface Forum. 2023, MPI: A Message-Passing Interface Standard, Version 4.1, `https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf`, approved by the MPI Forum on 2023 November 2
Morel, R., Rochette, G., Leonarduzzi, R., Bouchaud, J.-P., & Mallat, S. 2025, Applied and Computational Harmonic Analysis, 75, 101724
Mousset, L., Allys, E., Price, M. A., et al. 2024, A&A, 691, A269
Nocedal, J. & Wright, S. J. 2006, Numerical Optimization, 2nd edn., Springer Series in Operations Research and Financial Engineering (New York: Springer)
Paszke, A., Gross, S., Massa, F., et al. 2019, in Advances in Neural Information Processing Systems, ed. H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, & R. Garnett, Vol. 32 (Curran Associates, Inc.)
Petri, A., Haiman, Z., Hui, L., May, M., & Kratochvil, J. M. 2013, Phys. Rev. D, 88, 123002
Pineau, F.-X. 2026, cdshealpix: the CDS HEALPix library in Rust, `https://github.com/cds-astro/cds-healpix-rust`, rust crate `cdshealpix`, version 0.10.1 (`https://crates.io/crates/cdshealpix`); Centre de Données astronomiques de Strasbourg
Portilla, J. & Simoncelli, E. P. 2000, International Journal of Computer Vision, 40, 49
Price, I., Sanchez-Gonzalez, A., Alet, F., et al. 2025, Nature, 637, 84
Regaldo-Saint Blancard, B., Allys, E., Boulanger, F., Levrier, F., & Jeffrey, N. 2021, A&A, 649, L18
Schmalzing, J. & Górski, K. M. 1998, MNRAS, 297, 355
Smith, B. F., Bjørstad, P. E., & Gropp, W. D. 1996, Domain Decomposition: Parallel Multilevel Methods for Elliptic Partial Differential Equations (Cambridge: Cambridge University Press)
Steinbusch, B., Gaspar, A., & Brown, J. 2026, rsmpi: MPI bindings for Rust, `https://github.com/rsmpi/rsmpi`, rust crate `mpi`, version 0.8.2 (`https://crates.io/crates/mpi`)
Stevens, B., Satoh, M., Auger, L., et al. 2019, Progress in Earth and Planetary Science, 6, 61
STL-Proj. 2025, STL-Dev: Scattering Transform Library (development repository), `https://github.com/STL-Proj/STL-Dev`, public GitHub repository of the STL-Proj organisation, created in December 2025; documentation at `https://stl-dev.readthedocs.io`; accessed 2026 October 27
Takahashi, R., Hamana, T., Shirasaki, M., et al. 2017, ApJ, 850, 24
Tomita, H. 1986, Progress of Theoretical Physics, 76, 952
Villani, C. 2009, Grundlehren der mathematischen Wissenschaften, Vol. 338, Optimal Transport: Old and New (Berlin, Heidelberg: Springer)
Virtanen, P., Gommers, R., Oliphant, T. E., et al. 2020, Nature Methods, 17, 261
Wolfe, P. 1969, SIAM Review, 11, 226
Zonca, A., Singer, L. P., Lenz, D., et al. 2019, Journal of Open Source Software, 4, 1298

Appendix A: Minkowski functionals on HEALPix

For a map u with mean removed and standard deviation σ_0, $x = u/\sigma_0$, the Minkowski functionals of the excursion sets $\{x > \nu\}$ on the unit sphere are (Schmalzing & Górski 1998)

$$V_0(\nu) = \frac{1}{4\pi}\int \Theta(x-\nu)\,\mathrm{d}\Omega, \tag{A.1}$$

$$V_1(\nu) = \frac{1}{4\pi}\int \frac{1}{4}\,\delta(x-\nu)\,|\nabla x|\,\mathrm{d}\Omega, \tag{A.2}$$

$$V_2(\nu) = \frac{1}{4\pi}\int \frac{1}{2\pi}\,\delta(x-\nu)\,\frac{2x_{;\theta}x_{;\phi}x_{;\theta\phi} - x_{;\theta}^2 x_{;\phi\phi} - x_{;\phi}^2 x_{;\theta\theta}}{x_{;\theta}^2 + x_{;\phi}^2}\,\mathrm{d}\Omega, \tag{A.3}$$

with the covariant derivatives $x_{;\theta} = \partial_\theta x$, $x_{;\phi} = \partial_\phi x/\sin\theta$, $x_{;\theta\theta} = \partial_\theta x_{;\theta}$, $x_{;\theta\phi} = \partial_\theta x_{;\phi}$ and $x_{;\phi\phi} = \partial_\phi x_{;\phi}/\sin\theta + \cot\theta\, x_{;\theta}$. V_2 is the integral of the geodesic curvature of the contours; by the Gauss–Bonnet theorem it differs from the Euler characteristic of the excursion set by an area term, and its Gaussian expectation has no curvature correction on the sphere. For a Gaussian field with $\tau = \langle|\nabla x|^2\rangle/2$ (Tomita 1986),

$$V_0 = \tfrac{1}{2}\mathrm{erfc}\left(\nu/\sqrt{2}\right), \quad V_1 = \tfrac{1}{8}\sqrt{\tau}\,e^{-\nu^2/2}, \quad V_2 = \frac{\tau\,\nu\,e^{-\nu^2/2}}{(2\pi)^{3/2}}, \tag{A.4}$$

and $\tau = \sum_\ell (2\ell+1)\ell(\ell+1)C_\ell/\left(2\sum_\ell(2\ell+1)C_\ell\right)$.

`tools/minkowski.py` computes the derivatives in harmonic space: `healpy.alm2map_der1` gives $x_{;\theta}$ and $x_{;\phi}$ from the harmonic coefficients of the (optionally smoothed) map, and applied to the harmonic coefficients of these two maps it gives the second derivatives. The Dirac distribution is estimated by the fraction of pixels in bins of width $\Delta\nu = 0.2$. The test of the tool (`tools/test_minkowski.py`) draws Gaussian random fields at $N_{\mathrm{side}} = 256$ with a white spectrum smoothed over 4 pixels and compares them with the expressions above, for the τ measured on each field. The largest deviation per field is 0.2, 1.1 and 3.1% of the maximum of V_0, V_1 and V_2; for the mean of four fields 0.1, 0.4 and 1.3%; the measured τ agrees with the spectrum's to 0.06%. A lognormal field of the same spectrum is flagged (deviations of 71% and 100% of the maxima of V_1 and V_2). For the convergence maps we use thresholds $-3 \le \nu \le 6$ after a Gaussian smoothing over 4 pixels of the map's resolution and over 30 arcmin.